

MINI-PROJECT 2

Virtual Memory

Prepared by

Group: มูลนิธิทนายตั้ม ช่วยสู้

Guntee Doungmanee	67070501003
Drarun Yookphan	67070501016
Natthawat Primsirikunawut	67070501027
Rattathamnoon Buranapattana	67070501037
Wisit Suwannao	67070501042
Supawit Marayat	67070501045
Polwarit Watthanahemmarat	67070501067

Submitted to

Asst. Prof. Rajchawit Sarochawikasit

This report is part of CPE333 Operating Systems

Department of Computer Engineering, Faculty of Engineering

King Mongkut's University of Technology Thonburi

Semester 1, Academic Year 2026

Contents

Problem 1: Project scope and design **1**

Step 1: Virtual memory and page faults 1

Step 2: How the tool works 3

Problem 2: Experimental method and results **6**

Step 1: Test environment and method 6

Step 2: Results 9

Step 3: Post-merge checks 13

Problem 3: Limitations and conclusion **15**

Step 1: Limitations 15

Step 2: Conclusion 16

References **17**

Appendix: Reproduction steps and evidence **18**

Step 1: Check out the tested source 18

Step 2: Create and prepare the Lima VM 18

Step 3: Build and load the module 18

Step 4: Inspect a live process 19

Step 5: Run the demo and regression checks 19

Step 6: Run experiments E1 to E8 19

Step 7: Unload and remove build artifacts 19

Problem 1: Project scope and design

The project is a Linux tool for inspecting a process's **memory regions**, **virtual address space**, and **minor and major page faults**. It combines the `pfcount` kernel module, the `pfstat` command-line program, and the `faultgen` workload generator. We tested the merged PR #11 source on Ubuntu 24.04.5 with Linux 6.8.0-142-generic on ARM64. The build, demo, regression suite, and experiments E1 to E8 all completed in that environment.

These measurements describe different things. A process can reserve a large virtual range before its pages become resident, and file cache, readahead, fault-around, and Transparent Huge Pages (THP) can all change the recorded fault counts. The results describe the tested kernel configuration; they do not establish compatibility with other kernels or measure probe overhead.

Table 1: *Project requirements and how we checked them*

Requirement	Implementation	Check
Select a process	`pfstat PID` writes the target PID to `/proc/pfcount/target`	Read the report for a stable process
Count memory regions	Walk the process's VMAs	Compare with `/proc/PID/maps`
Measure virtual address space	Sum `vm_end - vm_start` for every VMA	Compare with maps, `mm->total_vm`, and VmSize
Count minor and major faults	Read lifetime counters and count live faults with a kretprobe	Compare with kernel counters using the same process scope
Explain Virtual Memory behavior	Relate VMAs, demand paging, cache, and THP to E1 to E8	Analyze the measured results in Problem 2, Step 2

The required output is the four process-level measurements listed above. Watch mode, run mode, descendant tracking, RSS reporting, and exit-time snapshots are additional features. We tested the tool in one Linux VM with one kernel configuration, described in Problem 2, Step 1.

Step 1: Virtual memory and page faults

Virtual address space and VMAs

A process uses a **virtual address space** that Linux describes through `struct mm_struct`. The kernel divides it into **Virtual Memory Areas (VMAs)**. Each VMA is a contiguous address range with common permissions and backing information. Its start and end addresses are `vm_start` and `vm_end`, so its size is their difference [3].

A VMA may contain many pages. The number of VMAs therefore is not the number of pages or the amount of resident memory. Summing the VMA sizes gives the mapped virtual address space. Comparing this value with RSS shows how much address space is

mapped and how much memory is resident. Experiment E7 uses both measurements to illustrate demand paging.

Page faults and fault classification

With **demand paging**, Linux prepares a page when a process first accesses it. A **minor fault** can be handled without storage I/O. A **major fault** requires I/O to obtain the page [4]. These counters record kernel fault events. They do not count load/store instructions or device I/O requests.

The module reads the return value from `handle_mm_fault()` and applies the classification rules used by `mm_account_fault()` in the tested kernel. A `VM_FAULT_RETRY` result is not counted until the final attempt completes. A `VM_FAULT_ERROR` result is recorded as a failed fault. A successful result is major when `VM_FAULT_MAJOR` is set or the original flags contain `FAULT_FLAG_TRIED`; otherwise, it is minor [2, 5].

```
if (ret & VM_FAULT_RETRY)
    /* wait for the final attempt */;
else if (ret & VM_FAULT_ERROR)
    /* record a failed fault */;
else if ((ret & VM_FAULT_MAJOR) || (flags & FAULT_FLAG_TRIED))
    /* count a major fault */;
else
    /* count a minor fault */;
```

The assignment suggests the `exceptions:page_fault_user` tracepoint. That tracepoint belongs to the x86 page-fault path, while our test VM is ARM64. It also fires before the kernel has finished handling the fault, so it does not directly provide the information needed to distinguish minor from major faults [2, 5]. We instead use a kretprobe on `handle_mm_fault()` and classify the completed result. This choice matches the Linux 6.8 accounting rules used in the test environment. We did not measure the probe's performance overhead systematically.

Page cache, readahead, and huge pages

A file-backed mapping can use the **page cache** without a new device read. We separated cold-cache and warm-cache cases and used `MADV_RANDOM` in selected tests to reduce readahead. On the tested kernel, disabling readahead did not disable fault-around. Linux could still map nearby pages that were already in the cache.

Transparent Huge Pages (THP) can also change fault counts. The `faultgen` program requests THP with `MADV_HUGEPAGE` and ordinary pages with `MADV_NOHUGEPAGE`. These calls advise the kernel but do not guarantee that every mapping receives a huge page [6]. The fault counts therefore describe the workload; on their own, they do not prove that every requested mapping used THP.

Step 2: How the tool works

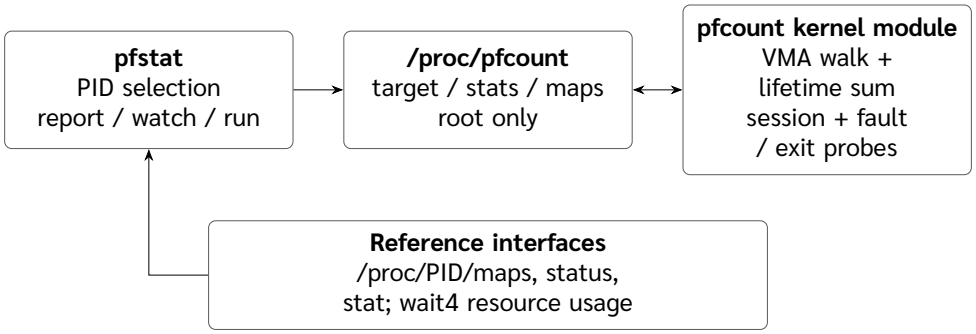


Figure 1: *User-space tools, kernel data sources, and comparison interfaces*

Counting regions and virtual address space

When a user selects a PID, `pfstat` writes it to `/proc/pfcount/target` and reads the module's stats and maps interfaces. The module obtains a reference to the process address space with `get_task_mm()`, takes `mmap_read_lock_killable()`, and walks the VMAs with `VMA_ITERATOR` and `for_each_vma`. It then releases the lock and the address-space reference with `mmput()` [2, 3].

For each VMA, the module increments the region count and adds `vm_end - vm_start` to the virtual size. It also groups mappings as file-backed, anonymous, heap, stack, or special mappings. Those groups help explain the output; they do not change the definition of a region. The module's total is checked against the address ranges in `/proc/PID/maps`, `mm->total_vm`, and VmSize.

```
mm = get_task_mm(task);
mmap_read_lock_killable(mm);
VMA_ITERATOR(vmi, mm, 0);
for_each_vma(vmi, vma) {
    regions++;
    vsize += vma->vm_end - vma->vm_start;
}
mmap_read_unlock(mm);
mmput(mm);
```

This snippet shows the main sequence. The implementation also handles a missing task, a task without an address space, and an interrupted lock acquisition.

Lifetime and live fault counters

Lifetime counters combine `min_flt` and `maj_flt` values from live threads with totals transferred to `signal_struct` when threads exit. The reviewed source uses `sig->stats_lock` and a sequence/retry pattern like `/proc/PID/stat`. This avoids counting a thread twice

or reading it while its totals are being transferred [2, 7]. These counters cover the thread group from process start.

Live counters start when the user selects a target. On entry to the `handle_mm_fault()` kretprobe, the module checks whether the faulting process is in scope and saves the flags and backing type for that invocation. On return, it uses the completed result to update minor, major, anonymous, file-backed, or error counters. Keeping entry data with its invocation prevents the module from classifying a fault from a CPU error code before the kernel has handled it [2, 5, 8].

Target sessions and process identity

Each target selection creates a new `pf_session` with a PID reference, start time, and its own counters. The probe entry handler holds a reference to the session that accepted a fault until the return handler finishes. A fault that began under an earlier target therefore cannot increment the new target's counters. RCU publication and reference counting let an in-flight handler finish safely while the target changes.

The module compares the thread-group `struct pid`, not only the numeric TGID. The kernel may reuse a PID after reaping the original process. Holding a reference to the PID object keeps the module from treating a new process with the same number as the original target. Problem 2, Step 3 checks this behavior with the PID-reuse regression.

Exit snapshots and permissions

For a short-lived process that exits before the user reads its stats, the module captures the region count, virtual size, and peak size through a kprobe on `acct_collect(exitcode, group_dead)`. The kernel calls this function after `atomic_dec_and_test()` identifies the last thread and before `exit_mm()`, while the address-space counters are still available. The probe uses `group_dead` rather than guessing that a thread is last from `signal->live == 1` at the start of `do_exit()` [2, 9].

The module sets permissions to 0600 for `/proc/pfcount/target` and 0400 for `stats` and `maps`. The interface can inspect the address space of a PID that the caller could not otherwise read, so the files are restricted to root. Our tests invoked the tool with `sudo`.

Command-line modes

Table 2: Main pfstat modes

Command	Behavior
`sudo tools/pfstat PID`	Print one summary and compare it with procfs
`sudo tools/pfstat PID --maps`	Include the process's VMA list
`sudo tools/pfstat PID -w 0.5`	Sample repeatedly and calculate rates using elapsed time
`sudo tools/pfstat -- CMD`	Run a command and count faults from before exec until exit
`sudo tools/pfstat -f -- CMD`	Include descendants that remain within the tracked ancestry

In run mode, the parent stops the child with SIGSTOP, selects it as the target, reads the lifetime baseline, and then allows it to exec the requested command. The program subtracts that baseline before comparing its live counters with `wait4()`. The child drops back to the invoking user's privileges before exec. After fork, the program restores the inherited SIGCHLD disposition in the child and uses a cleanup path to reset the target and follow state if execution or waiting fails. The post-merge test also covers a parent that inherited `SIGCHLD=SIG_IGN`.

The terminal report below is a separate example for the current shell. It shows the VMA total, virtual size, fault counters, and mapping groups in one view; the E1 measurements that follow use the recorded `sleep` and PID 1 runs.

```
pfcount report: pid 1622 (bash)
  Memory regions (VMAs)                    36   match vs /proc/PID/maps
  Total virtual address space           8.7 MB   match vs /proc/PID/maps
    mm->total_vm                        8.7 MB   match vs VmSize
    peak (VmPeak)                       8.7 MB
  Resident set size                     4.8 MB
  Minor page faults                     3,397   match vs /proc/PID/stat
  Major page faults                         2   match vs /proc/PID/stat
  Threads                                   1

  Regions by type           count          size
    file                       28        6.9 MB
    anon                        4        108 KB
    heap                        1        1.5 MB
    stack                       1        132 KB
    special                     2         12 KB
```

Figure 2: Example `pfstat` report for the shell, including procfs comparisons and VMA groups.

In this shell example, the report finds 36 VMAs and 8.7 MB of virtual space. Both values match the mapping ranges in the process's `maps` file. `mm->total_vm` matches VmSize, and 3,397 minor and 2 major faults match the corresponding `stat` counters. The final rows group the VMAs by backing type.

Problem 2: Experimental method and results

Step 1: Test environment and method

Table 3: *Post-merge test environment*

Item	Value
Host and virtual machine	macOS host; Lima VM with 4 reported vCPUs and 3,905 MB guest memory
Guest and kernel	Ubuntu 24.04.5 LTS; Linux 6.8.0-142-generic; aarch64
Compiler	GCC 13.3.0
Page size and THP	4,096 bytes; THP mode set to `madvise`
Fault-around and readahead	65,536 bytes and 128 KiB, respectively
Kernel options	`CONFIG_KPROBES=y`, `CONFIG_KRETPROBES=y`, `CONFIG_BSD_PROCESS_ACCT=y`
Source revision	PR #11 merge commit `5fbcb767`

The saved environment record identifies the guest kernel, required kernel options, and exact merge commit used for the post-merge measurements. The capture below isolates the three required kernel options so they remain readable at print size; the kernel version and architecture are recorded in the terminal output above.

```
commit: 5fbcb7679298f36e2bae8b7cd3dbe6bf6a609800
Linux lima-cpe333 6.8.0-142-generic #142-Ubuntu SMP PREEMPT_DYNAMIC Wed Sep 2
    14:30:29 UTC 2026 aarch64 aarch64 aarch64 GNU/Linux
CONFIG_BSD_PROCESS_ACCT=y
CONFIG_KPROBES=y
CONFIG_KRETPROBES=y
```



Figure 3: *Terminal output confirming the kernel options required by the module.*

We kept raw outputs and environment details in the `evidence/` directory so the post-merge measurements remain separate from earlier PR tests. The program labels sizes

as MB and KB but uses a base of 1,024. This report uses MiB and KiB when stating exact sizes; for example, `anon:64` means 64 MiB.

Build and load

Inside Ubuntu, `vm.sh` prepared the Lima VM and installed build tools and headers matching `uname -r`. The build and load command, `make clean && make all && make load`, completed and created `/proc/pfcount/maps`, `/proc/pfcount/stats`, and `/proc/pfcount/target`.

The build reported a compiler-name mismatch despite GCC 13.3.0, shared-folder clock skew, and skipped BTF because `vmlinux` was unavailable. Kbuild still completed and the module loaded. Other kernels and architectures were not tested.

```
./vm.sh setup
./vm.sh make clean
./vm.sh make all
./vm.sh make load
./vm.sh make demo
./vm.sh make experiments
./vm.sh run sudo sh tests/regress.sh
./vm.sh make unload
./vm.sh make clean
```

The saved build/load log records the object compilation, module link, and proc entries. Workspace-specific path prefixes are omitted here.

```
make -C module
  CC [M] pfcount.o
  MODPOST Module.symvers
  LD [M] pfcount.ko
  BTF [M] pfcount.ko
Test binaries compiled: faultgen, threadexit, threadchurn, uffdblock
/proc/pfcount/: maps, stats, target
```

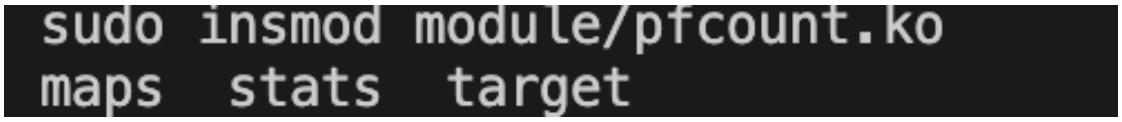


Figure 4: *Terminal output confirming that the loaded module exposes its proc interfaces.*

End-to-end demo

The demo command was `sudo tools/pfstat -w 0.5 -- tests/faultgen` with the workload phases `sleep:1 anon:64 sleep:1 file:/tmp/pfcount-big.bin sleep:1 free sleep:1`. The complete interval samples, rates, workload checkpoints, and exit summary below come from the saved terminal output in `evidence/post-merge-5fbcb76/02-demo-terminal-capture.txt`. The `faultgen` checkpoints are cumulative from process start, while `pfstat` subtracts the baseline recorded before `exec`; in this run that excludes 472 earlier minor faults. The resulting 16,782 minor and 4,097 major faults match the baseline-adjusted `getrusage()` comparison.

```
[faultgen 1935] start                          minflt=601 majflt=1
[faultgen 1935] anon 64 MB (16384 pages)       minflt=16998 majflt=1
[faultgen 1935] file 32768 KB (8192 pages)     minflt=17254 majflt=4097
[faultgen 1935] free                           minflt=17254 majflt=4097
[faultgen 1935] done                           minflt=17254 majflt=4097
    time  regions       vsize         rss      minflt    majflt     min/s    maj/s
     0.0        1        4 KB        0 KB         564         0         0        0
     0.5       16      2.2 MB      1.4 MB         614         1        99        2
     1.0       17     66.2 MB      9.1 MB       2,593         1     3,915        0
     1.5       17     66.2 MB     65.4 MB      16,998         1    28,668        0
     2.0       17     66.2 MB     65.4 MB      16,998         1         0        0
     2.5       18     98.2 MB     97.2 MB      17,254     4,097       511    8,173
     3.0       18     98.2 MB     97.2 MB      17,254     4,097         0        0
     3.5       16      2.2 MB      1.4 MB      17,254     4,097         0        0
     4.0       16      2.2 MB      1.4 MB      17,254     4,097         0        0

pfcount run: tests/faultgen ... (pid 1935, 4.54s, exit 0)
  Memory regions at exit                    16
  Virtual address space at exit          2.2 MB
  Peak virtual address space            98.2 MB
  Minor page faults                     16,782   match vs getrusage()
  Major page faults                      4,097   match vs getrusage()
    on anonymous memory                 16,454
    on file-backed memory                4,425
  fault scope: process only
```

```
  Memory regions at exit                    16
  Virtual address space at exit          2.2 MB
  Peak virtual address space            98.2 MB
  Minor page faults                     16,649   match vs getrusage()
  Major page faults                      6,145   match vs getrusage()
    on anonymous memory                 16,450
    on file-backed memory                6,344
  fault scope: process only
```

Figure 5: *Terminal capture from a separate demo run. Fault totals vary with cache state; the latest transcript above supplies the values used in the analysis.*

How we checked the results

We compared region counts and address-space sizes using a process that remained stable while the module and procfs were read. In run mode, we compared the final live counters with the child usage returned after exit, after subtracting the lifetime baseline. Agreement between these interfaces supports the observed behavior in the tested cases; both still rely on counters maintained by the same Linux kernel.

E1 compared the report with procfs. E2 to E6 varied anonymous mappings, file cache, readahead, and THP. E7 recorded virtual size and RSS over time and then unmapped the regions. E8 compared ordinary programs with and without descendant tracking. The post-merge regression suite covered retargeting, PID reuse, simultaneous thread exit, lifetime counter reads, module unload during a blocked fault, and cleanup after inherited SIGCHLD settings. Permission checks and interval parameters were present in earlier PR evidence; the post-merge suite summarized here did not rerun the non-root permission case.

```
== e1a-report-sleep: report vs /proc on sleep(1)
== e1b-report-init: report vs /proc on PID 1
== e2-anon: 64 MB anonymous, 4 KB pages
== e3-filecold: 32 MB file, cold cache, MADV_RANDOM
== e4-filewarm: 32 MB file, warm cache, MADV_RANDOM
== e5-fileseq: 32 MB file, cold cache, readahead on
== e6-thp: 64 MB anonymous, transparent huge pages
== e7-timeline: timeline: anon, file, free
== e8a-python: python3 -c pass
== e8b-ls: sh -c ls, process only
== e8c-ls-follow: sh -c ls, with descendants
== e8d-gcc: gcc driver, process only
== e8e-gcc-follow: gcc with cc1/as/ld descendants
results in results/
```

Figure 6: *Terminal summary of the E1 to E8 cases run by the experiment script. Numeric results appear in the following tables and discussion.*

Step 2: Results

E1: comparison with procfs

Table 4: *E1 results from merge commit 5fbcb767*

Target	Regions	Virtual size	Minor faults	Major faults
`sleep`	30	5.6 MB	106	0
PID 1, `systemd`	113	21.9 MB	12,986	94

The saved E1 logs also contain the mapping breakdown. These condensed output records omit region addresses and keep the cross-checks and VMA groups together:

```
sleep (pid 10605): 30 VMAs, 5.6 MB; 106 minor, 0 major
  maps, VmSize, and /proc/PID/stat comparisons: match
  file 24/5.2 MB; anon 2/56 KB; heap 1/132 KB; stack 1/132 KB; special 2/12 KB
systemd (pid 1): 113 VMAs, 21.9 MB; peak 22.0 MB; RSS 12.1 MB
  12,986 minor, 94 major; maps, VmSize, and /proc/PID/stat comparisons: match
  file 98/18.5 MB; anon 11/356 KB; heap 1/2.9 MB; stack 1/132 KB; special 2/12
   KB
```

For both targets, the region count and summed address ranges matched `maps`. The module's `mm->total_vm` matched VmSize, and its lifetime fault counts matched `stat` in

the recorded reads. PID 1 provided a system-process check under root. Its fault count changes with system activity, so these values describe the recorded run rather than a fixed property of PID 1.

E2 to E6: mapping type and memory settings

Table 5: *Live fault counts compared with wait4() after baseline subtraction*

Test	Workload	Minor	Major	Comparison
E2	Anonymous mapping, 64 MiB	16,538	1	Match
E3	Cold file, 32 MiB, RANDOM	143	8,193	Match
E4	Warm file, 32 MiB, RANDOM	652	1	Match
E5	Cold file, 32 MiB, readahead	655	2	Match
E6	Anonymous mapping, 64 MiB, THP request	188	1	Match

E2 wrote once to each ordinary page in a 64 MiB anonymous mapping. At a 4 KiB page size, the mapping contains 16,384 pages. The 16,538 minor faults also include process startup activity while monitoring was active, so the process-wide total is not expected to equal the number of intentional writes exactly.

E3 read a 32 MiB file, or 8,192 pages, after requesting that its data be dropped from the page cache. The program used `MADV_RANDOM`. It recorded 8,193 major faults, including one that occurred before the file read. The increase during the read is consistent with 8,192 pages; major faults do not directly count device I/O requests.

E4 read a file already in cache. It produced mostly minor faults, and far fewer file faults than the 8,192 pages in the file. This result is consistent with fault-around mapping multiple nearby cached pages per fault. The configured `fault_around_bytes=65536` spans 16 ordinary pages. E5 enabled readahead for a cold file and recorded 2 major faults, compared with 8,193 in E3. Together, these runs show that fault counts depend on cache state, readahead, and fault-around, not only file size.

E6 requested THP for a 64 MiB anonymous mapping and recorded substantially fewer minor faults than E2. This is consistent with a fault mapping more memory at a time. If the kernel backs the whole region with 2 MiB huge pages, it would use about 32 such pages, but startup and other activity remain in the process-wide count. We did not collect `AnonHugePages` for every mapping, so the fault count alone does not confirm that all requested pages became huge pages.

The saved run summaries also show how the counts split between anonymous and file-backed memory. Each pair below matched the corresponding `getrusage()` result.

```
E2 anon:64       minflt=16,538  majflt=1  anon=16,462  file-backed=77
E3 filecold      minflt=143     majflt=8,193  anon=71  file-backed=8,265
E4 filewarm      minflt=652     majflt=1  anon=68  file-backed=585
E5 fileseq       minflt=655     majflt=2  anon=72  file-backed=585
E6 thp:64        minflt=188     majflt=1  anon=112  file-backed=77
fault scope: process only
```

E7: virtual size and RSS over time

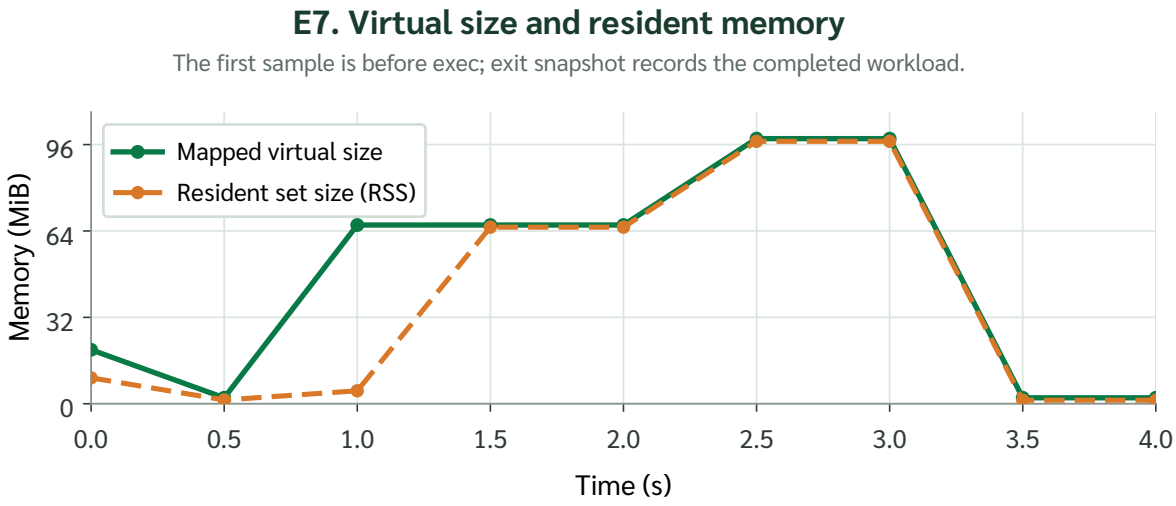


Figure 7: *Virtual size and RSS recorded by the E7 command-line timeline*

When the program created an anonymous mapping, virtual size grew from about 2.2 MiB to 66.2 MiB before RSS reached the same level. At 1.0 second, RSS was 4.8 MiB. After the program touched the pages, RSS rose to 65.4 MiB. Mapping and reading a 32 MiB file increased the totals to about 98.2 MiB virtual size and 97.2 MiB RSS. After `munmap`, both returned close to their earlier levels. This run shows that reserving a range does not immediately make all of its pages resident in RAM.

The first watch-mode sample can occur between fork and exec or while the address space is changing. A sampled maximum should therefore not be treated as the child program's peak without checking the exit-time snapshot. The module reads the peak from the mm counters when the process exits. The reported minor/second and major/second rates use the counter difference divided by elapsed time between samples.

This is the terminal record used for the timeline figure and the values above. It includes the full sample sequence and exit-time totals.

```
$ sudo tools/pfstat -w 0.5 -- tests/faultgen sleep:1 anon:64 sleep:1 file:/tmp/
    pfcount-big.bin sleep:1 free sleep:1
[faultgen 10637] start                          minflt=602 majflt=1
[faultgen 10637] anon 64 MB (16384 pages)       minflt=16999 majflt=1
[faultgen 10637] file 32768 KB (8192 pages)     minflt=16999 majflt=8193
[faultgen 10637] free                           minflt=16999 majflt=8193
[faultgen 10637] done                           minflt=16999 majflt=8193
    time  regions       vsize         rss      minflt    majflt     min/s    maj/s
     0.0       51     20.0 MB      9.6 MB         564         0         0        0
     0.5       16      2.2 MB      1.4 MB         615         1       101        2
     1.0       17     66.2 MB      4.8 MB       1,486         1      1741        0
     1.5       17     66.2 MB     65.4 MB      16,999         1     30871        0
     2.0       17     66.2 MB     65.4 MB      16,999         1         0        0
     2.5       18     98.2 MB     97.2 MB      16,999     8,193         0    16212
     3.0       18     98.2 MB     97.2 MB      16,999     8,193         0        0
     3.5       16      2.2 MB      1.3 MB      16,999     8,193         0        0
     4.0       16      2.2 MB      1.3 MB      16,999     8,193         0        0

pfcount run: tests/faultgen sleep:1 anon:64 sleep:1 file:/tmp/pfcount-big.bin sleep
    :1 free sleep:1  (pid 10637, 4.52s, exit 0)
  Memory regions at exit                    16
  Virtual address space at exit          2.2 MB
  Peak virtual address space            98.2 MB
  Minor page faults                     16,524   match vs getrusage()
  Major page faults                      8,193   match vs getrusage()
    on anonymous memory                 16,453
    on file-backed memory                8,264
  fault scope: process only
```

E8: process scope and descendants

Table 6: *E8 results under different fault scopes*

Command	Minor	Major	Reference comparison
`python3 -c pass`	1,234	0	Matches `getrusage()`
`sh + ls`, without `-f`	258	0	135 below `wait4()`
`sh + ls`, with `-f`	390	0	Matches `wait4()`
`gcc`, without `-f`	287	0	5,700 below `getrusage()`
`gcc`, with `-f`	5,971	0	Matches `wait4()`

The shell and GCC launch other processes. In process-only mode, `pfstat` counts the selected process but not its children. The reference returned by `wait4()` can include usage from waited children. In these runs, enabling follow mode brought the `ls` and GCC results into agreement with their waited-descendant references. Python did not create a child

and matched `getrusage()` in process-only mode. The differences in the table reflect the scope used by each counter; they do not by themselves show a counting error.

`wait4()` does not always include every descendant. A child may be auto-reaped when its parent sets `SIGCHLD` to `SIG_IGN`. A child reparented before it is waited for may also be absent from the reference [4, 9]. The CLI therefore labels the reference as waited descendants only and explains when the live count exceeds it. The post-merge cleanup test covers the inherited `SIGCHLD=SIG_IGN` case.

The saved E8 outputs make the scope difference visible for the same shell and compiler workloads:

```
$ sudo tools/pfstat -- sh -c 'ls -R /usr/include >/dev/null'
  Minor faults 258; 135 below getrusage() because child usage is excluded
$ sudo tools/pfstat -f -- sh -c 'ls -R /usr/include >/dev/null'
  Minor faults 390; matches wait4() for waited descendants
$ sudo tools/pfstat -- gcc -O2 -o /tmp/fg tests/faultgen.c
  Minor faults 287; 5,700 below getrusage() because child usage is excluded
$ sudo tools/pfstat -f -- gcc -O2 -o /tmp/fg tests/faultgen.c
  Minor faults 5,971; matches wait4() for waited descendants
```

Step 3: Post-merge checks

We reviewed and tested the source at merge commit `5fbcb767`; the measurements below come from that revision, not from a pre-merge branch. The original review focused on target changes during faults, PID reuse, concurrent thread exit, lifetime-counter aggregation, and session cleanup during module unload.

Table 7: *Build, demo, and workflow checks after merge*

Case	Check	Observed result
Build and load	`make clean && make all && make load`	Passed; all proc entries were created
Demo	`make demo`; compare faults with `getrusage()`	Exit status 0; minor and major counts matched
Experiments	Run E1 to E8 with `scripts/experiments.sh`	Exit status 0; E8 scope differences were explained
Cleanup	Run `make clean` after testing	Passed; test binaries were removed

The post-merge regression output is shown alongside these checks. It records the PID-reuse, concurrency, lifetime-counter, and unload cases completed by the suite.

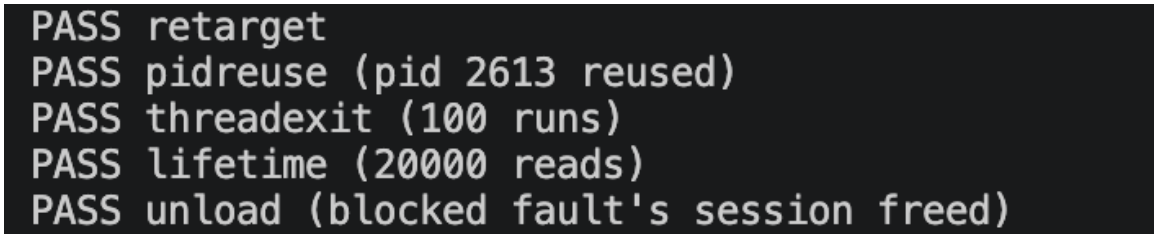


Figure 8: *Terminal results from the post-merge lifecycle and concurrency regression suite.*

Table 8: *Lifecycle and concurrency regression checks after merge*

Case	Check	Observed result
Retarget	Generate faults in A, then select stopped process B	Passed; A's faults did not enter B's session
PID reuse	Reuse the numeric PID of an earlier target	Passed; old target remained exited and its counters did not change
Concurrent thread exit	End a thread group concurrently	Exit snapshots were complete in 100 runs
Lifetime aggregation	Read counters while creating and exiting threads	20,000 reads; no counter decrease observed
Unload during a blocked fault	Unload while `userfaultfd` held a fault	Passed; canceled-probe sessions were released during unload
Inherited SIGCHLD	Run `pfstat -f -- /bin/true` with parent `SIGCHLD=SIG_IGN`	Passed; child exited 0, target reset to idle, follow reset to N

The merged source retains a reference to the session that accepted each fault until the return handler completes. It identifies targets by `struct pid`, uses `group_dead` to detect the final thread, and protects lifetime totals with `stats_lock`. It also releases sessions left by canceled probes during module unload [2]. In run mode, the program temporarily sets SIGCHLD to the default disposition while waiting and restores the child's inherited disposition before exec. The post-merge test confirmed that `wait4()` still collects the child status and that cleanup returns the target and follow state to idle and N.

The source sets `0600` on `/proc/pfcount/target` and `0400` on `stats` and `maps`. A non-root permission check appears in the pre-merge evidence, but we did not rerun that case after merge, so it is not listed as a post-merge regression result.

The inherited SIGCHLD cleanup check also captured the process summary and the final idle state of the module:

```
pfcount run: /bin/true  (pid 15578, 0.01s, exit 0)
  Memory regions at exit                    15
  Virtual address space at exit          2.1 MB
  Peak virtual address space             2.2 MB
```

```
  Minor page faults                          122   match vs wait4(), waited
    descendants only
  Major page faults                            0   match vs wait4(), waited
    descendants only
     on anonymous memory                      60
     on file-backed memory                    62
  fault scope: process + descendants
pid: 0
state: idle
follow=N
```

GitHub Actions checks were not configured for the branch we reviewed. The build and regression results in this report came from manual runs inside Lima, not from an automated GitHub workflow [10].

Problem 3: Limitations and conclusion

Step 1: Limitations

The results apply to Ubuntu 24.04.5, Linux 6.8.0-142-generic, ARM64, and the kernel options in Table 2. Without `CONFIG_BSD_PROCESS_ACCT=y`, the module cannot register the `acct_collect()` probe and therefore cannot capture exit snapshots. It can still report a running target, but it cannot confirm final values for a short-lived process [2]. VMA iteration and probe arguments also depend on kernel APIs, so other versions and architectures need separate builds and checks.

Live counters begin when a target is selected. Descendant tracking follows `real_parent` ancestry up to 64 levels, and reparenting can change which processes are in scope. `wait4()` reports usage passed through waited children; auto-reaped or reparented children may be absent. Comparisons must use the same time window and process scope.

The kretprobe runs on the page-fault path before filtering the target and uses `maxactive=64`. If more concurrent invocations occur than the probe can handle, the kernel increments `nmissed` [8]. The recorded runs showed no reported probe misses, but we did not measure overhead under sustained fault load. A follow-up measurement should compare the same workload with the module unloaded, loaded without a target, and loaded with a target.

The module, maps, status, and stat interfaces are read at different times. A process that changes mappings or creates threads during a comparison can produce differences due to timing. Stable or paused targets make structural comparisons easier to interpret. The report also retains byte-level values before converting them to rounded display units.

Results from E3 to E6 depend on page-cache state, readahead, fault-around, and whether the kernel accepts the THP advice. `POSIX_FADV_DONTNEED` and `MADV_HUGEPAGE` do not guarantee a fixed outcome. We did not collect `AnonHugePages` for every mapping, so the THP result is suggestive rather than proof that all pages were backed by huge pages.

The comparisons with `/proc/PID/stat`, `getrusage()`, and `wait4()` are consistency checks against counters maintained by the same Linux kernel. They are not independent external measurements. The tests did not cover another kernel version, x86_64, another PID namespace, or a systematic performance benchmark.

Step 2: Conclusion

The tool accepts a PID and reports the four values required by the assignment. After merge, the region count and virtual size matched procfs for the recorded targets, and the page-fault counters matched kernel references when the process scope and time window aligned. E2 to E7 also showed that virtual size can grow before RSS and that page-cache state, readahead, fault-around, and THP requests affect observed fault counts.

The build, demo, regression suite, and experiments E1 to E8 passed in the documented Lima environment. These results support the implementation for the tested workloads and configuration. They do not establish compatibility with every Linux kernel or architecture, and the probe overhead remains unmeasured.

References

[1] CPE333 Mini-Project 2 assignment, Academic Year 2026, pp. 2-5.
../CPE333_MiniProject2_2026.pdf

[2] beambeambeam/cpe333, Mini-Project 02 source after PR #11 was merged.
https://github.com/beambeambeam/cpe333/tree/5fbcb7679298f36e2bae8b7cd3dbe6bf6a609800/mini-project-02

[3] The Linux Kernel documentation, Process Addresses.
https://docs.kernel.org/mm/process_addrs.html

[4] Linux man-pages, `getrusage(2)`, resource usage and child accounting.
https://man7.org/linux/man-pages/man2/getrusage.2.html

[5] Linux v6.8 source, `mm/memory.c`: `handle_mm_fault()` and `mm_account_fault()`.
https://github.com/torvalds/linux/blob/v6.8/mm/memory.c

[6] Linux man-pages, `madvise(2)`, mapping and huge-page advice.
https://man7.org/linux/man-pages/man2/madvise.2.html

[7] Linux v6.8 source, `fs/proc/array.c`: `do_task_stat()` and `stats_lock`.
https://github.com/torvalds/linux/blob/v6.8/fs/proc/array.c

[8] The Linux Kernel documentation, Kernel Probes.
https://docs.kernel.org/trace/kprobes.html

[9] Linux v6.8 source, `kernel/exit.c`: `do_exit()`, `group_dead`, and waited-child accounting.
https://github.com/torvalds/linux/blob/v6.8/kernel/exit.c

[10] PR #11 review discussion and revisions.
https://github.com/beambeambeam/cpe333/pull/11

Appendix: Reproduction steps and evidence

The report uses the merged source at commit `5fbcb767`. The source code and its build, experiment, and regression scripts are in the team's Mini-Project 2 source folder. The steps below reproduce the recorded Linux run from a macOS host with Homebrew and Lima.

Step 1: Check out the tested source

Clone the team repository and select the exact merge commit used for the report. This keeps later changes on `main` from changing the source under test.

```
git clone https://github.com/beambeambeam/cpe333.git
cd cpe333/mini-project-02
git checkout 5fbcb7679298f36e2bae8b7cd3dbe6bf6a609800
```

Step 2: Create and prepare the Lima VM

Run the project helper from the `mini-project-02/` directory. If Lima is not installed, the helper installs it with Homebrew. It creates an Ubuntu 24.04 VM, mounts the project directory read-write, and installs the compiler, Python, and headers for the guest kernel.

```
./vm.sh setup
./vm.sh run uname -a
```

The tested guest used Ubuntu 24.04.5, Linux 6.8.0-142-generic, and ARM64. The module needs `CONFIG_KPROBES=y`, `CONFIG_KRETPROBES=y`, and `CONFIG_BSD_PROCESS_ACCT=y`; the recorded values are in `evidence/post-merge-5fbcb76/00-environment.txt`.

Step 3: Build and load the module

Run these commands from the source directory on the macOS host. The helper executes each Make target inside the VM. `make all` builds the module and the `faultgen` test program; `make load` inserts the module and checks that its proc entries exist.

```
./vm.sh make clean
./vm.sh make all
./vm.sh make load
./vm.sh run ls -l /proc/pfcount
```

The module exposes three interfaces under `/proc/pfcount`:

```
/proc/pfcount/target
/proc/pfcount/stats
/proc/pfcount/maps
```

The recorded build skipped BTF because `vmlinux` was unavailable; compilation and module loading still completed.

Step 4: Inspect a live process

Open a VM shell, start a stable process, then compare `pfstat --maps` with `/proc/PID/maps`.

```
./vm.sh shell
sleep 300 &
PID=$!
sudo tools/pfstat "$PID" --maps
sudo cat /proc/"$PID"/maps
kill "$PID"
exit
```

Check that the VMA count and summed ranges match `/proc/PID/maps`; compare virtual size with `VmSize` and lifetime faults with `/proc/PID/stat`.

Step 5: Run the demo and regression checks

The demo watches `faultgen`, then compares exit-time counts with `getrusage()`. The regression suite covers retargeting, PID reuse, thread exit, lifetime-counter reads, and unload during a blocked fault.

```
./vm.sh make demo
./vm.sh run sudo sh tests/regress.sh
```

The latest demo transcript and regression log are in `evidence/post-merge-5fbcb76/02-demo-terminal-capture.txt` and `evidence/post-merge-5fbcb76/03-regression.txt`; the demo screenshot is a separate, labeled run.

Step 6: Run experiments E1 to E8

The script rebuilds helpers, reloads the module, runs the procfs comparison and controlled workloads, records output in `results/`, then unloads the module.

```
./vm.sh make experiments
```

E1 cross-checks procfs; E2 to E6 vary mapping type, cache state, readahead, and THP advice; E7 samples virtual size and RSS while the workload maps and frees memory; E8 compares process-only with descendant tracking. The report explains each result, and raw outputs are in `evidence/post-merge-5fbcb76/experiment-results/`.

Step 7: Unload and remove build artifacts

Unload the module, remove generated binaries, and stop Lima. These commands preserve the source and saved evidence.

```
./vm.sh make unload
./vm.sh make clean
./vm.sh stop
```

The evidence bundle contains environment, build/load, demo, regression, cleanup, and E1 to E8 records, plus six Terminal captures. Full output remains in searchable text files.